



OBJECT ORIENTED PROGRAMMING

(23EN1202)

UNIT- 4: Part 1

FILE HANDLING METHODS

Topics



What is file?

- A **file** is a collection of data stored on a secondary storage device (like a hard disk).
- Data on non-volatile storage media is stored in **named locations**, which are called **files**.
- Example: Like a notebook → we can **open, read, write, and close** it.
- Using the input() function for large amounts of data is tedious.
- A **better option** is to use **files** for storing and processing data.

File-Contd.

S. No.	Volatile Memory	Non-Volatile Memory
1.	<u>Volatile memory</u> is the type of memory in which data is lost as it is powered-off.	<u>Non-volatile memory</u> is the type of memory in which data remains stored even if it is powered-off.
2.	Contents of Volatile memory are stored temporarily.	Contents of Non-volatile memory are stored permanently.
3.	It is faster than non-volatile memory.	It is slower than volatile memory.
4.	<u>RAM</u> (Random Access Memory) is an example of volatile memory.	<u>ROM</u> (Read Only Memory) is an example of non-volatile memory. Hard disk, USB drive, DVD etc

File-Contd.

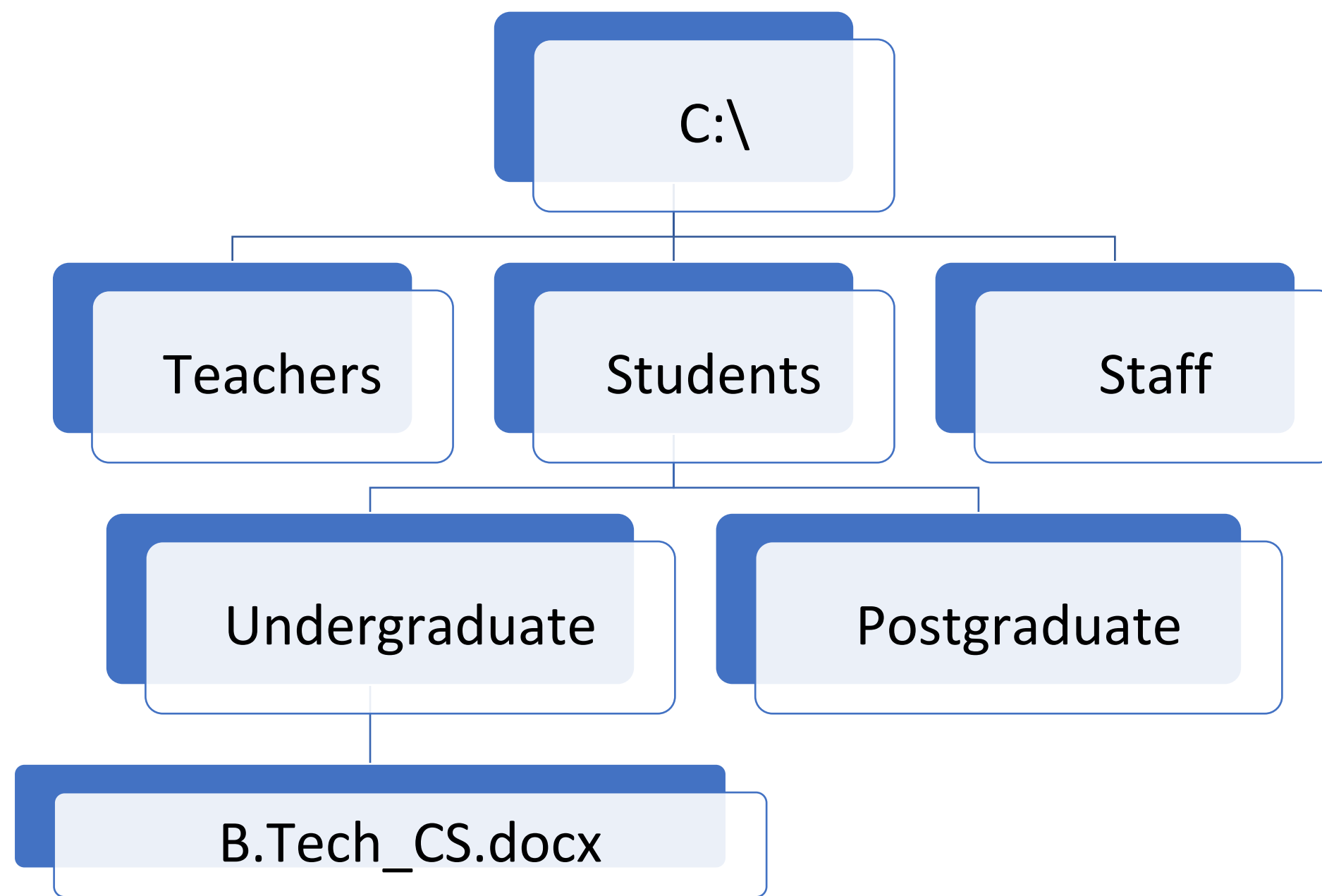
Problems with console oriented I/O operations:

- Time consuming to handle huge amount of data.
- Using terminal, entire data is lost when the program is terminated or computer is turned off.

FILE PATH

- Files are stored in a tree or hierarchical structure.
- The top of the tree is one root nodes.
- Under root node, there are other files and folders (directories).
- Each folder contain other files and folders, It is a limitless path.
- Type of the file is indicated by its extension.
- **Two types of paths:** Absolute pathname, Relative pathname

Files and Folders



Absolute Pathname

Always contains the root and the complete directory list to specify the exact location of the file.

- Ex: C:\students\undergraduate\Btech_CS.docx

Relative Pathname

Starts with respect to the current working directory and therefore lacks the leading slashes.

- Ex: undergraduate\Btech_CS.docx

FILE TYPES



Text Files



Binary Files

Text Files

Text files are simple texts in human readable format.

It is stream of characters that are sequentially processed by a computer.

It can be opened for reading, writing or appending.

Each line contains zero or more characters and maximum of 255 characters.

In text file each line is terminated by special character called EOL.

Text Files

Text files don't have any specific encoding and it can be opened in normal text editor itself.

- **Web standards:** html, XML, CSS, JSON etc.
- **Source code:** c, app, js, py, java etc.
- **Documents:** txt, tex, RTF etc.
- **Tabular data:** csv, tsv etc.
- **Configuration:** ini, cfg, reg etc.

Binary Files

It stores the information in the same format as in the memory i.e. data is stored according to its data type so no translation occurs.

In binary file there is no delimiter for a new line.

Binary files are faster and easier for a program to read and write than text files.

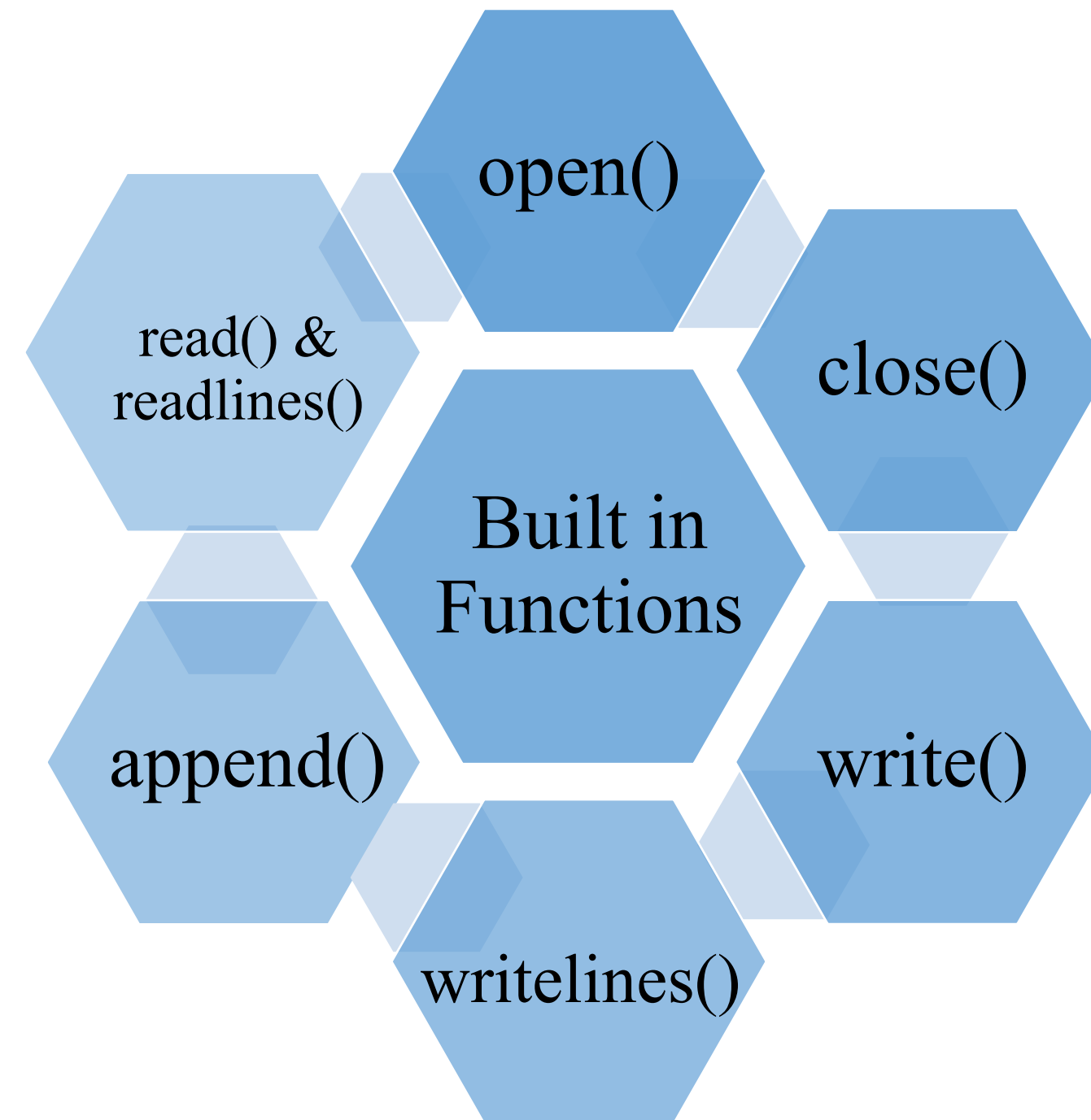
Data in binary files cannot be directly read, it can be read only through python program for the same

Binary Files

Most of the files that we see in our computer system are called binary files.

1. **Document files:** .pdf, .doc, .xls etc.
2. **Image files:** .png, .jpg, .gif, .bmp etc.
3. **Video files:** .mp4, .3gp, .mkv, .avi etc.
4. **Audio files:** .mp3, .wav, .mka, .aac etc.
5. **Database files:** .mdb, .accde, .frm, .sqlite etc.
6. **Archive files:** .zip, .rar, .iso, .7z etc.
7. **Executable files:** .exe, .dll, .class etc.

File Operations



The Open() Function

- Python has a built-in `open()` function to open files from the directory.
- Two arguments that are mainly needed by the `open()` function are:
- file name or file path and mode in which the file is opened.
- The syntax for opening a file is

```
file_object = open(file_name , access_mode)
```


open()

- **file_name**: File name contains a string type value containing the name of the file which we want to access.
- **access_mode**: The value to access_mode specifies the mode in which we want to open the file, i.e., read, write, append, etc.

Example for open()

Opening a file in current directory

*first create any file with “.txt” extension in same folder.

Example:

```
file=open("test.txt","rb")
```

```
print(file)
```

Output:

```
<_io.BufferedReader name='test.txt'>
```

Example for open()

Opening a file in different directory

```
f=open("G:/DSU/CTPY/test.txt")
```

```
f
```

Opens in w mode(writing only)

```
f=open("test.txt",'w')
```

```
f
```

output: <_io.TextIOWrapper name='test.txt' mode='w' encoding='cp1252'>

#read and write in binary mode

```
f=open("test.txt", 'rb+')
```

```
f
```

output: <_io.BufferedRandom name='test.txt'>

Modes	Description
r	It opens a file in reading mode. The file pointer is placed at the starting of the file. It is the default mode.
rb	It opens a file in reading only mode in binary format. The file pointer is placed at the starting of the file.
r+	It opens the file in both reading and writing mode. The file pointer is placed at the starting of the file.
rb+	It opens the file in both reading and writing mode in binary format. The file pointer is placed at the starting of the file.
w	It opens the file in writing only mode. If a file exists, it overwrites the existing file; otherwise, it creates a new file.
wb	It opens the file in writing only mode in binary format. If a file exists, it overwrites the existing file; otherwise, it creates a new file.
w+	It opens the file in booth reading and writing mode. If a file exists, it overwrites the existing file; otherwise, it creates a new file.
wb+	It opens the file in booth reading and writing mode in binary format. If a file exists, it overwrites the existing file; otherwise, it creates a new file.
a	It opens a file for appending. The file pointer is placed at the end of the file. If the file does not exist in the directory, it creates a new file for writing.
ab	It opens a file for appending in binary format. The file pointer is placed at the end of the file. If the file does not exist in the directory, it creates a new file for writing.
a+	It opens a file for appending and reading. The file pointer is placed at the end of the file. If the file does not exist in the directory, it creates a new file for writing.
ab+	It opens a file for appending and reading in binary format. The file pointer is placed at the end of the file. If the file does not exist in the directory, it creates a new file for writing.

Closing a File

- When the operations that are to be performed on an opened file are finished, we have to close the file in order to release the resources.
- Although Python comes with a garbage collector responsible for cleaning up the unreferenced objects from the memory, we must not rely on it to close a file.

Closing a File

- Proper closing of a file frees up the resources held with the file.
- The closing of file is done with a built-in function `close()`.

Syntax:

```
fileObject.close()
```

Closing a File

Example 1

open a file

```
f = open("test.txt",'wb')
```

perform file operations

close the file

```
f.close()
```

f

File Object Attributes

- Once a file is opened and a file object created
- Various pieces of information about that file can be gathered by using some predefined attributes.

Attribute	Description
<code>file.closed</code>	It will return <i>True</i> if the file is closed; it will otherwise return <i>False</i> .
<code>file.mode</code>	It will return the access mode with which the file is opened.
<code>file.name</code>	It will return the name of the file.
<code>file.softspace</code>	It will return <i>False</i> if space explicitly required with print; otherwise it will return <i>True</i> .

Writing in a File

- open it with 'w' mode, or 'a' mode, or any writing-enabling mode.
- We should be careful when using the w mode because in this mode overwriting persists in case the file already exists.

Syntax

```
fileobject.write(string)
```

Example

open the file with w mode

```
f = open("test_write.txt",'w')
```

perform write operation

```
f.write('Hello All 1\n')
```

```
f.write('Welcome to DSU 2\n')
```

```
f.write('This is our Python class 3\n')
```

```
f.write('It is an interesting programming language')
```

close the file after writing

```
f.close()
```

 jupyter test_write.txt ✓ a few seconds ago

File Edit View Language

```
1 Hello All 1
2 Welcome to DSU 2
3 This is our Python class 3
4 It is an interesting programming language
```

Reading from File

- Must open the file in the reading mode (r mode).
- We can use read(size) method to read the data specified by size.
- If no size is provided, it will end up reading to the end of the file.
- Reading from any file is done with the method read().
- The read() method enables us to read the strings from an opened file.

Example

#Reading from a file

```
f = open("test_write.txt",'r')
```

```
f.read(5)
```

```
f.read(7)
```

```
f.read()
```

readline()

- readline() is a method of reading a file line by line. Whenever we write fileobject.readline().
- it prints one line from that file and continues in this way until the end of the file.

```
f = open("test_write.txt",'r')  
f.readline()
```


Opening Files using with Keyword

➤ When the file is opened using `with` keyword, file will be automatically closed once the block of code finishes execution.

➤ Ex:

```
with open("test.txt","r") as file:  
    for line in file:  
        print(line)  
print("lets check if the file is closed:",file.closed)
```

Output:
hello
python
jupiter
dsu
programming
lets check if the file is closed: True

Using open()

- When the file is opened using **open() function**, file will not be automatically closed.

➤ Ex:

```
f11=open("test1.txt","r")  
for lines in f11:  
    print(lines)  
print("lets check if the file is closed:",f11.closed)
```

Output:

jupiter

moon

sun

lets check if the file is closed: **False**

File Position

- When we read a line or some data from a file, the pointer points to the next line or data, and that when we end up reading whole file, it returns the empty string.
- In Python, the `tell()` method tells us about the current position of the pointer.
- The current position tells us where reading will start from at present.

File Position

- We can also change the position of the pointer with the help of `seek()` method.
- Syntax: `seek(offset[,from])`
- Offset-A number of bytes are passed to be moved by the pointer as arguments to the `seek()` method.
- from-is given in table

Table 7.4 From and its Position

From	Reference Position
0	From the beginning of the file
1	From the current position of the file
2	From the end of the file

Example of File position

open the file

```
f = open("test_write.txt",'r')
```

read 28 bytes of data

```
f.read(28)
```

check the current position

```
f.tell()
```

change the position to beginning

```
f.seek(0)
```

again read 25 bytes

```
f.read(25)
```

Renaming a File

- Renaming a file in Python is done with the help of the `rename()` method.
- The `rename()` method is passed with two arguments, the current filename and the new filename.

Syntax

```
os.rename(current_filename, new_filename)
```

OS Module

- The OS module in Python provides functions for interacting with the operating system.
- OS comes under Python's standard utility modules.
- This module provides a portable way of using operating system-dependent functionality.
- The `*os*` and `*os.path*` modules include many functions to interact with the file system.

Deleting File

- Deleting a file in Python is done with the help of the `remove()` method.
- It takes the filename as an argument to be deleted.

Syntax

```
os.remove(filename)
```


Example

```
# import os
```

```
>>>import os
```

```
# deleting the file
```

```
>>>os.remove("test1.txt")
```

Methods to Manipulate Files

Sr. No.	Methods with Description
1.	<code>file.close()</code> After performing operation, it closes the file.
2.	<code>file.flush()</code> It flushes the internal buffer memory.
3.	<code>file.fileno()</code> It returns the integer file descriptor.
4.	<code>file.isatty()</code> It returns True if file is connected with tty(-like) device, False otherwise.
5.	<code>file.next()</code> It returns the next line from the file.

Methods to Manipulate Files

6.	<code>file.read([size])</code> Reads the size bytes from a file.
7.	<code>file.readline([size])</code> It reads the entire one line from a file.
8.	<code>file.readlines([sizehint])</code> It reads until the end of the file using <code>readline</code> . It returns the list of lines read.
9.	<code>file.seek([offset])</code> It changes the current position.
10.	<code>file.tell()</code> It returns the file's current position.
11.	<code>file.truncate([size])</code> It truncates the file.
12.	<code>file.write(str)</code> It writes the <code>str</code> string to the file.
13.	<code>file.writelines(sequence)</code> It writes the sequence of strings into a file. If each string in the sequence should go into separate lines in file, the string should end with a new line character, <code>'\n'</code> .

Activity

- ❑ *Write a function in python to read the content from a text file "introduction.txt" line by line and display the same on screen.*
- ❑ *Create a text file and write some contents and few lines will start with letter 'M'. Write a function in python to count the number of lines from a text file "introduction.txt" which is not starting with an alphabet "M".*

Solution

```
def read_file():  
    with open("introduction.txt", "r") as file:  
        for line in file:  
            print(line, end=" ") # end="" avoids extra blank lines  
  
# Call the function  
read_file()
```

Solution

```
def count_M_lines():  
    count = 0  
    with open("introduction.txt", "r") as file:  
        for line in file:  
            if line.startswith('M'): # check if line begins with M  
                count += 1  
    print("Number of lines starting with M:", count)  
  
# Call the function  
count_M_lines()
```

Activity

- ❑ *Write a function in Python to read lines from a text file “introduction.txt”. Your function should find and display the occurrence of the word “is”.*

- ❑ *Create a text file and store some contents in that file. Display the contents of the text which needs to be displayed such that every next character is separated by a symbol “*”. Write a function definition for that it would display the entire content of the file in the desired format.*

UNIT- 4: Part 2

USER DEFINED CLASSES AND OBJECTS

CLASSES AND OBJECTS

- A **class** is a **blueprint** or **template**, using which **objects** are created.
- Class defines the **structure** and **behavior** that objects of that type will possess. This includes data members also known as **attributes** and member functions also known as **methods** that operate on that data.
- When a class is "**instantiated**," an object is created based on the specifications defined by the class. Each object has its own unique set of attribute values, but shares the same methods as other objects of the same class. Hence an **object** can be visualised as an **instance** of a **class**.

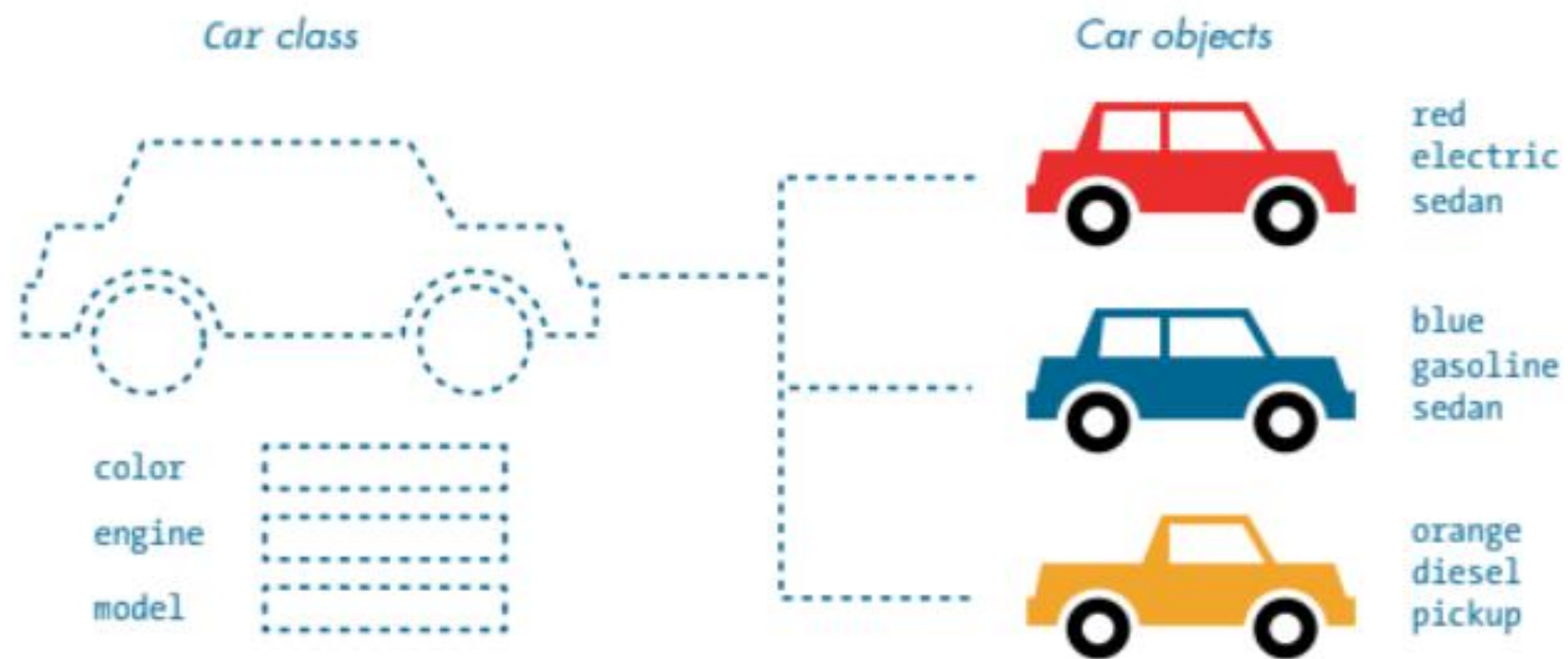
INTRODUCTION

- Procedure-oriented programming
 - Program is written using **functions** or blocks of statements to manipulate data.
- Object oriented programming
 - Object Oriented Programming is a programming paradigm based on the concept of **objects**.
 - Objects contain data, in the form of attributes, and actions, in the form of functions known as methods.
 - The aim of object oriented programming is to reuse the same code by providing the flexibility to create each object with their own features.

Example

- If "**Car**" is a **class**, it might define **attributes** like "**color**," "**make**," and "**model**," and **methods** like "**startEngine()**" or "**accelerate()**."
- An object, like "**myCar**," would be an instance of the "**Car**" class, with specific values for its attributes (e.g., "red," "Toyota," "Camry").
- In Python, everything is an object or an instance of some class. For example, all integer variables are instances of class *int* and all string variables are instances of class *str*.

Example



DEFINING A CLASS

- A class is created with the keyword *class* followed by the class name.
- Within the indentation of the *class* block, write the class members i.e., **attributes** and **actions**.
- The attributes are represented by **variables** and actions are performed by **methods**.
- Class members are accessed through **objects** of that particular class.

DEFINING A CLASS

- **Naming Convention**

- Each word of a class name should start with a **capital letter**.
- When a class represents exception, then its name should end with the word '**error**'.

- **Note:**

- Class definition can appear anywhere in the program.
- Usually class definition is written near the beginning of the program, after the *import* statement.

DEFINING A CLASS

- A class creates a new local namespace where all its members are defined. Namespace defines the scope of members specific to the class which means all attributes (variables) and methods (functions) defined within the class body belong to this class's local namespace. They are distinct from names in other namespaces (like the global module namespace or other class namespaces). This local namespace prevents name conflicts. For example, if you have a variable named name inside a class and another name variable in the global scope, they are treated as separate entities.
- A class that has no statements should have a *pass* statement at least.

DEFINING A CLASS

Example 1

```
# creating the class  
class Person():  
    pass
```

Example 2

```
class Car():  
    pass
```


CREATING OBJECTS

- Once a class is created, the next job is to create an object or instance of that class.
- The class members are accessed through the class object using the dot (.) operator.
- Creating an object or instance of a class is known as *class instantiation*.
- The code shown in the next slide creates an empty object of the class Person.
- When an instance of the class (object) is created, the instance name holds the memory address of the instance.

CREATING OBJECTS

Example 1

```
# creating the class
class Person():
    pass
```

```
# Create an instance of the Person class and store into the variable saathvik
saathvik = Person()
```

```
print(saathvik)
```

```
<__main__.Person object at 0x000001FEF4CE1610>
```

Note: The above result shows that an instance was built from Person class and the instance is stored in memory location.

CREATING MULTIPLE INSTANCES

- We can create as many instances as we want from each class.
- Store each instance in a separate variable.
- **Note:** Although the two instances are created from the same class, they are stored as separate entities within the program.

Example 1

```
# creating the class
class Person():
    pass
```


CREATING MULTIPLE INSTANCES

```
# Create an instance of the Person class and store into the variable samhithaa
samhithaa = Person()

print(samhithaa)
```

```
<__main__.Person object at 0x000001FEF4CE26C0>
```

```
# Create an instance of the Person class and store into the variable suchethana
suchethana = Person()

print(suchethana)
```

```
<__main__.Person object at 0x000001FEF4D10FE0>
```

EXERCISE

- Create a class called "Animals", and create 2 instances from it, namely, "lion" and "tiger".

```
class Animals():  
    pass
```

```
lion = Animals()  
  
print(lion)
```

```
<__main__.Animals object at 0x000001FEF39629F0>
```

```
tiger = Animals()  
  
print(tiger)
```

```
<__main__.Animals object at 0x000001FEF3860CB0>
```

DATA ABSTRACTION

- **Data abstraction** is a process by which data and functions are defined in such a way that only essential details are provided to the outside world and the implementation details are **hidden**.
- In Python, a class provides **methods** to the outside world to provide the functionality of the object or to manipulate the object's data.

DATA ENCAPSULATION

- **Encapsulation** involves the **bundling** of data members and functions inside a single class.
- Encapsulation defines different **access levels** for data variables and member functions of the class:
 - **public**
 - Any data or function with access level public can be accessed by any function belonging to any class.
 - **private**
 - Any data or function with access level private can be accessed only by the class members, in which it is declared.
 - In Python, private variables are prefixed with a double underscore.
 - Example: `__date_of_birth = None`

METHOD

- Objects are associated with certain **attributes** and **actions** with them.
- Example:
 - Consider a car. It has attributes like make, model, variant, color, number of wheels, etc.
 - It also perform actions like accelerate, stop, turn, etc.
- In a class, actions are represented through **methods**.
- Methods are essentially functions defined in the class.

DEFINING AND CALLING A METHOD

- To define a method, define a function within the *class* indentation block.
- To call the method use **dot syntax** and use parenthesis after the name.
- Naming Convention
 - Method name should be all **lower case letters**.
 - When multiple words are used for a name, separate them using an underscore.

DEFINING AND CALLING A METHOD

Example 1

```
# Defining and calling a method
class Dog():
    def bark():
        print('Bow! Bow!')
```

```
Dog.bark()
```

```
Bow! Bow!
```

THE CLASS CONSTRUCTOR

- A **constructor** is a special method that is used to **initialize** the instance variables of a class.
- A constructor is a method that has 2 underscores before and after the word *init*:
 - **`__init__()`**
- A constructor includes the “**self**” variable as a **first mandatory parameter**.
- A constructor facilitates to instantiate objects with different attribute values upon creation.
- The **`__init__()`** method is executed automatically when an object of a class is created.

THE CLASS CONSTRUCTOR

Example

```
# Create a Person class with init method
class Person():
    def __init__(self):
        pass
```

SELF PARAMETER

Note:

- The *self* argument is ignored while the *init* method is called. The *init* method is called immediately when an object is created.
- It is recommended to initialize all the attributes of a class in the `__init__()` method.

SELF PARAMETER

Example

```
# Create a Person class with init method
class Person():
    def __init__(self):
        self.name = 'Deepak'
        self.gender = 'Male'
```

```
deepak = Person()
```

```
deepak.name
```

```
'Deepak'
```

DESTRUCTOR

- **Destructor** is automatically called when an object is going **out of scope**.
- Destructor is used to return the resources occupied by the object, back to the system so that they can be reused.
- In Python, `__del__()` method does the job of the destructor.
- **Note:**
 - If the method is intended to be accessed through instance, use *self* as the first parameter in the definition. Without the *self* parameter, the method can only be accessed by the class itself.
 - A class can have as many methods as required.

DESTRUCTOR

```
class Car():  
    def __init__(self):  
        self.speed = 0  
  
    def __del__(self):  
        print("The object is out of scope.")
```

```
ioniq = Car()
```

```
del ioniq
```

```
The object is out of scope.
```


VARIABLES

- The **attributes** of an object are represented through **variables** in a class.
- The variables written inside a class are of 2 types:
 - **Class** Variables or **Static** variables
 - **Instance** Variables
- Naming Convention:
 - Variable names should be all **lower case letters**.
 - When multiple words are used, separate them using an **underscore**.

CLASS OR STATIC VARIABLES

- **Class variables** are the variables whose **single copy** is available to all the instances of the class.
- Class variables are also called **static variables**.
- Class variables are accessed outside the class using the class name followed by the dot operator.

CLASS OR STATIC VARIABLES

Example

```
class Account():  
    interest_rate = 0.075
```

```
print(Account.interest_rate)
```

0.075

CLASS OR STATIC VARIABLES

- To access the class variables, use class methods.
- If we modify the class variable in an instance, it will be modified in all the other instances.

Example

```
class Number():  
    num = 10  
  
    @classmethod  
    def increment(cls):  
        cls.num += 1
```

```
num_one = Number()  
num_two = Number()  
  
print(num_one.num)  
print(num_two.num)
```

```
10  
10
```

```
num_one.increment()
```

```
print(num_one.num)
```

```
print(num_two.num)
```

```
11  
11
```

INSTANCE VARIABLES

- **Instance variables** are the variables whose **separate copy** is created in every instance.
- Instance variables are defined and initialized using the **constructor**.
- Instance variables are accessed using **instance methods**.

INSTANCE VARIABLES

Example

```
class Person():  
    def __init__(self):  
        self.name = 'Deepak'  
        self.gender = 'Male'
```

```
deepak = Person()
```

```
print(deepak.name)
```

Deepak

INSTANCE VARIABLES

```
print(Person.name) # Try accessing object variable like class variable
```

```
-----  
AttributeError                                Traceback (most recent call last)  
Cell In[31], line 1  
----> 1 print(Person.name) # Try accessing object variable like class variable  
  
AttributeError: type object 'Person' has no attribute 'name'
```

EXERCISES

- 1. Create a Dog class that has one class attribute and two instance level attributes. The class attribute should be “species” with a value of “Canine.” The two instance attributes should be “name” and “breed.” Then instantiate two dog objects, a Husky named Sammi and a Chocolate Lab named Casey.
- 2. Create a Person class that has a single instance level attribute of “name.” Ask the user to input their name, and create an instance of the Person class with the name they typed in. Then print out their name.

PUBLIC AND PRIVATE DATA MEMBERS

- Public members can be accessed from anywhere in the program i.e., they can be accessed within the class as well as from outside the class in which they are defined.
- Private members can only be accessed within the class.
- Private members are defined in the class with a **double underscore prefix**.

PUBLIC AND PRIVATE DATA MEMBERS

Example 1 - Class Variables

```
class Car():  
    __authentication_key = 20240229  
  
    make = 'kia'  
  
    model = "Sonet"
```

```
print(Car.make, Car.model)
```

Kia Sonet

```
print(Car.__authentication_key)
```

```
-----  
AttributeError                                Traceback (most recent call last)  
Cell In[38], line 1  
----> 1 print(Car.__authentication_key)  
  
AttributeError: type object 'Car' has no attribute '__authentication_key'
```

PUBLIC AND PRIVATE DATA MEMBERS

Example 2 - Instance Variables

```
class Car():  
    def __init__(self):  
        self.__authentication_key = 20240229  
  
        self.make = 'Kia'  
  
        self.model = "Sonet"
```

```
my_kia = Car()
```

```
print(my_kia.make, my_kia.model)
```

Kia Sonet

```
print(my_kia.__authentication_key)
```

```
-----  
AttributeError                                Traceback (most recent call last)  
Cell In[44], line 1  
----> 1 print(my_kia.__authentication_key)  
  
AttributeError: 'Car' object has no attribute '__authentication_key'
```

```
print(my_kia._Car__authentication_key) # Not recommended
```

20240229

PRIVATE METHODS

- Private methods can't be accessed from anywhere outside the class.
- Private methods are used to include implementation details of an object.

```
class BankAccount():  
    def __get_atm_pin(self):  
        print("*****")
```

```
my_bank = BankAccount()
```

```
my_bank._BankAccount__get_atm_pin() # Not recommended
```

```
*****
```

CLASS METHODS

- Class methods are called by a class and not by instance of the class.
- The first parameter of the class methods is ***cls*** and **not *self***.

CLASS METHODS

```
class Rectangle():
    def __init__(self, length, breadth):
        self.length = length
        self.breadth = breadth

    def area(self):
        return self.length * self.breadth

    @classmethod
    def Square(cls, side):
        return cls(side, side)
```

```
s = Rectangle.Square(5)
```

```
print(s)
```

```
<__main__.Rectangle object at 0x000002161EA11F90>
```

```
s.area()
```

STATIC METHODS

- A functionality that belongs to a class, but does not require the object, is placed in the **static** method.
- Static methods does not receive any additional arguments like *self*, *cls*.
- A static method is defined using a built-in function *staticmethod*.
- Static method knows nothing about the class and just deals with the parameters.
- Static method cannot access the properties of the class itself.
- When we need a utility function that doesn't access any properties of a class but makes sense that it belongs to the class, we use static methods.
- A static method can be called either on the class or on an instance.

STATIC METHODS

Example

```
# Class to write application errors into a log file
```

```
class ErrorLog():  
    @staticmethod  
    def log_error(error):  
        with open("log.txt", "a") as log_file:  
            log_file.write("\n")  
            log_file.write(error)
```

✓ 0.0s

```
# Calling a static method using class name
```

```
ErrorLog.log_error("NameError: name 'colours' is not defined")
```

✓ 0.0s

+ Code

+ Markdown

```
# Calling a static method using instance name
```

```
log = ErrorLog()
```

```
log.log_error("KeyError: 'Semester'")
```

✓ 0.0s

EXERCISES

- Python program to manage patient records effectively in Hospital Management System using Data Abstraction.
- Python program to develop a University Management System to efficiently manage information related to students and faculty members while maintaining data security and proper encapsulation.
- Python program to develop a program to manage employee information in an organization using a constructor for data initialization.
- Design a Math Operations Utility class to perform basic arithmetic operations using static methods.